

DevOps Journey - Day 3: Shell Scripting

Suraj Rane

Day 3: Log Rotation & Disk-Space Alerting - Putting Days 1-2 Together

Today's about building something you'd actually find in a production environment - a script that manages log growth and warns before disks fill up. This pulls together functions, getopt, file reading, arrays, and exit-status handling from the last two days.

Part 1: Why Log Rotation Matters

Left unmanaged, log files grow forever and eventually fill the disk - which can crash an application or the whole server. Log rotation means: compress/archive old logs, delete logs past a retention period, and keep the "live" log file fresh.

Production systems usually use logrotate (a built-in Linux tool) for this - but writing your own version first is how you actually understand what it's doing under the hood.

1.1 Core Log Rotation Logic

```
rotate_log() {
    local logfile="$1"
    local max_size_kb="$2"

    if [ ! -f "$logfile" ]; then
        return 0
    fi

    local size_kb
    size_kb=$(du -k "$logfile" | cut -f1)

    if (( size_kb > max_size_kb )); then
        local timestamp
        timestamp=$(date '+%Y%m%d_%H%M%S')
        mv "$logfile" "${logfile}.${timestamp}"
        gzip "${logfile}.${timestamp}"
        touch "$logfile"
        log "Rotated $logfile (was ${size_kb}KB) ->
${logfile}.${timestamp}.gz"
    fi
}
```

Walking through it: `-du -k` gets the file size in KB - If it's over the threshold, rename it with a timestamp suffix (so old rotations don't overwrite each other) - `gzip` compresses the rotated file - `touch` recreates an empty live log file so the app can keep writing to it

1.2 Deleting Old Rotated Logs (Retention Policy)

```
cleanup_old_logs() {
    local log_dir="$1"
    local retention_days="$2"

    find "$log_dir" -name "*.gz" -mtime "+$retention_days" -exec rm
    {} \;
    log "Cleaned up logs older than $retention_days days in
$log_dir"
}
```

- `find ... -mtime "+7"` finds files last modified more than 7 days ago
- `-exec rm {} \;` deletes each matched file

This `find -mtime` pattern is extremely common in real ops scripts - worth memorizing.

Part 2: Disk-Space Alerting

2.1 Checking Disk Usage Per Mount Point

```
check_disk_usage() {
    local threshold="$1"
    local alert_triggered=false

    while IFS= read -r line; do
        local usage
        local mount
        usage=$(echo "$line" | awk '{print $5}' | tr -d '%')
        mount=$(echo "$line" | awk '{print $6}')

        if (( usage > threshold )); then
            log "ALERT: $mount is at ${usage}% (threshold:
${threshold}%)"
            alert_triggered=true
        fi
    done <<(df -h | tail -n +2)

    if [ "$alert_triggered" = true ]; then
        return 1
    fi
    return 0
}
```

2.2 New Concept: Process Substitution < <(...)

Notice `done <<(df -h | tail -n +2)` instead of a pipe. This is called **process substitution**. Recall from Day 2 - piping into a while loop runs it in a subshell, so variables set inside don't persist outside. Process substitution avoids that problem entirely, because the loop still reads from a file-like redirect (<), not a pipe.

```

# This runs in a subshell - alert_triggered set inside won't be
visible outside
df -h | tail -n +2 | while read -r line; do
    alert_triggered=true
done

# This does NOT run in a subshell - alert_triggered persists
correctly
while read -r line; do
    alert_triggered=true
done < <(df -h | tail -n +2)

```

This is the “proper fix” to the subshell gotcha - better than input redirection from a file when your data comes from a command instead of an actual file.

2.3 Sending an Alert (Email/Webhook)

In real environments you’d hook into email or Slack. A simple stub:

```

send_alert() {
    local message="$1"
    # Slack webhook example (replace with real webhook URL)
    # curl -X POST -H 'Content-type: application/json' \
    #     --data '{"text": "$message"}' "$SLACK_WEBHOOK_URL"

    # Or just log it for now / mail command if configured
    echo "$message" | wall 2>/dev/null || true
    log "ALERT SENT: $message"
}

```

Part 3: Putting It All Together - Script Structure

```

#!/bin/bash
set -euo pipefail

LOG_FILE="/var/log/log_manager.log"

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOG_FILE"
}

usage() {
    echo "Usage: $0 -d <log_dir> -s <max_size_kb> -r
<retention_days> -t <disk_threshold>"
    exit 1
}

rotate_log() {
    # ... (as above)
    :
}

cleanup_old_logs() {
    # ... (as above)
    :
}

```

```

}

check_disk_usage() {
    # ... (as above)
    :
}

main() {
    local log_dir="" max_size=1024 retention=7 threshold=80

    while getopts "d:s:r:t:" opt; do
        case $opt in
            d) log_dir="$OPTARG" ;;
            s) max_size="$OPTARG" ;;
            r) retention="$OPTARG" ;;
            t) threshold="$OPTARG" ;;
            \?) usage ;;
        esac
    done

    [ -z "$log_dir" ] && usage

    log "Starting log management for $log_dir"

    for logfile in "$log_dir"/*.log; do
        rotate_log "$logfile" "$max_size"
    done

    cleanup_old_logs "$log_dir" "$retention"

    if ! check_disk_usage "$threshold"; then
        send_alert "Disk usage threshold exceeded on $(hostname)"
        exit 1
    fi

    log "Log management complete"
    exit 0
}

main "$@"

```

3.1 New Thing Here: tee -a

tee -a "\$LOG_FILE" prints to the screen **and** appends to a file simultaneously - better than >> alone when you want to both see output live and keep a permanent record.

3.2 New Thing Here: main "\$@"

Passing "\$@" into main forwards all the script's original arguments into the function, so getopts inside main can process them. This is a very standard production pattern - keeps global scope clean and everything flows through main.

Part 4: Hands-On Assignment

Build log_manager.sh combining everything above:

1. Accepts flags: -d (log directory), -s (max size in KB before rotation), -r (retention days), -t (disk usage alert threshold)
2. Loops through all .log files in the given directory and rotates any that exceed the size threshold
3. Cleans up rotated (.gz) logs older than the retention period
4. Checks disk usage across mounted filesystems using process substitution (< <(...)) - not a pipe
5. If any mount exceeds the threshold, logs an alert and exits with code 1; otherwise exits 0
6. Test it: create a few dummy .log files of different sizes (dd if=/dev/zero of=test.log bs=1K count=1200 makes a ~1.2MB file) and confirm rotation actually triggers correctly

End of Day 3 - DevOps Journey (55-Day Roadmap)